



GATE PASS MONITING SYSTEM

SRS

GUIDE

DIVYA S NAIR

GROUP MEMBERS

MEENU V S

PAVITHRA J J

JITHU S SIVAN

RAHUL B

Gate Pass Monitoring System

1. Introduction

1.1 Purpose

The purpose of this SRS is to define the functional and non-functional requirements for the Gate Pass Monitoring System (GPMS). This document serves as a reference for developers, testers, stakeholders, and administrators involved in the system's design and implementation.

1.2 Scope

The Gate Pass Monitoring System is a web-based application developed using HTML, CSS, JavaScript (frontend) and Django with SQLite (backend). The system digitalizes and automates the process of issuing, approving, scanning, and tracking gate passes in an organization such as an educational institution.

It supports:

- Online gate pass requests
- Role-based approval workflow
- Barcode scanning for verification
- Entry/exit time logging
- Report generation

The system improves security, reduces paperwork, and enhances monitoring

1.3 Definitions

Term	Meaning
Gate Pass	Permission granted for exiting campus
Advisor	Requesting pass approving student requests
Security	Staff validating passes at gate

1.4 References

Project abstract — Gate Pass Monitoring System Using Web Development

Gate Pass

2. Overall Description

2.1 Product Perspective

The system replaces manual paper-based passes. It is a standalone web application hosted on a server, accessed through browsers. Barcode validation enables quick verification at entry/exit points.

2.2 Product Functions

- User registration & authentication
- Submit gate pass applications
- Role-based approval workflow:
 - Advisor → HOD → Security
- Unique barcode generation
- Barcode scanning at gate
- Entry & exit timestamp logging
- Dashboard & reports

2.3 User Classes

User Type	Description
Student	Requests gate pass
Class Advisor	Approves / rejects
HOD	Provides final approval
Security Officer	Scans barcode and logs movement
Admin	Manages system settings

2.4 Operating Environment

- Web Browser (Chrome, Firefox, Edge)
- Django Framework
- SQLite Database
- Windows / Linux Server

2.5 Constraints

- Internet connectivity required
- Secure authentication required
- Barcode scanner support required

2.6 Assumptions

- Users have valid credentials
- Device has working camera or scanner
- Institutional policies define approval rules

3. System Features

3.1 User Registration & Login

- Users must log in using valid credentials
- Password encryption enforced

3.2 Gate Pass Request

- Student submits:
 - Name
 - Admission No
 - Department
 - Batch / Year
 - Reason for leaving
 - Time & date
 - Phone contacts

3.3 Approval Workflow

- Advisor reviews request
- HOD gives final approval
- Status updated in real-time

3.4 Barcode Generation

- Each approved pass gets a unique barcode
- Barcode printed on ID card or displayed in portal

3.5 Barcode Scanning

- Security scans barcode
- System validates pass
- Logs:
 - Exit time
 - Entry time

3.6 Reports

- Daily movement logs
- Student history
- Unauthorized attempt logs
- Export support

4. External Interface Requirements

4.1 User Interface

- Simple web UI
- Dashboard views per role
- Color-coded status
- Mobile-friendly

4.2 Hardware Interface

- Laptop / PC
- Camera or barcode scanner

4.3 Software Interface

- Django backend
- SQLite database
- HTML/CSS/JS frontend

5. Non-Functional Requirements

5.1 Performance

- Handle minimum 500 users concurrently
- Response < 3 seconds

5.2 Security

- Encrypted passwords
- Role-based access control
- Session-based login
- Prevent:
 - SQL injection
 - XSS
- Unauthorized access

5.3 Availability

- 99% uptime target

5.4 Reliability

- Database backup support

5.5 Usability

- Minimal training required
- Intuitive UI

5.6 Scalability

- Supports future expansion

6. Database Requirements

- System must store:
 - Student details
 - Gate pass applications
 - Approval status
 - Logs
 - Reports
 - Primary keys must be unique.

7. Use Case Summary

Use Case 1 —Submit Gate Pass by scan barcode

- Actor: Advisor
- Outcome: Request submit Gate Pass

Use Case 2 — Scan Barcode

- Actor: Security
- Outcome: Entry/Exit logged

Use Case 3 — View the Gate Pass

- Actor: Advisor / HOD
- Outcome: Status updated

8. System Architecture (High Level)

- Client browser → Web server (Django) → SQLite DB
- Scanner communicates via web interface

9. Future Enhancements

- Mobile App
- OTP verification
- Push notifications
- AI-based behavior monitoring

10. Conclusion

The Gate Pass Monitoring System provides a secure, efficient, and scalable solution to replace manual gate pass processing while improving monitoring and security